```
import math
import random


def rastrigin(solution):
    dimensions = len(solution)
    return 10 * dimensions + sum(
        value ** 2 - 10 * math.cos(2 * math.pi * value)
        for value in solution
    )


def generate_neighbor(solution, bounds, step_size):
    candidate = solution.copy()

    index = random.randrange(len(candidate))

    candidate[index] += random.uniform(-step_size, step_size)

    lower, upper = bounds[index]
    candidate[index] = min(max(candidate[index], lower), upper)

    return candidate


def simulated_annealing(
    objective,
    bounds,
    step_size,
    initial_temp=100.0,
    cooling_rate=0.95,
    min_temp=1e-3,
    iterations_per_temp=100
):
    if not bounds:
        raise ValueError("At least one variable is required.")

    if step_size <= 0:
        raise ValueError("Step size must be positive.")

    if initial_temp <= min_temp or min_temp <= 0:
        raise ValueError(
            "Initial temperature must exceed minimum temperature.")

    if not 0 < cooling_rate < 1:
        raise ValueError("Cooling rate must be between 0 and 1.")

    if iterations_per_temp < 1:
        raise ValueError("Iterations per temperature must be positive.")

    if any(lower >= upper for lower, upper in bounds):
        raise ValueError("Each lower bound must be less than its upper bound.")

    current = [
        random.uniform(lower, upper)
        for lower, upper in bounds
    ]
    current_score = objective(current)

    best = current.copy()
    best_score = current_score

    temperature = initial_temp

    while temperature > min_temp:

        for _ in range(iterations_per_temp):

            candidate = generate_neighbor(
                current, bounds, step_size
            )
            candidate_score = objective(candidate)

            delta = candidate_score - current_score

            if delta <= 0 or random.random() < math.exp(
                -delta / temperature
            ):
                current = candidate
                current_score = candidate_score
```

```
                if current_score < best_score:
                    best = current.copy()
                    best_score = current_score

        temperature *= cooling_rate

    return best, best_score


bounds = [(-5.12, 5.12), (-5.12, 5.12)]


random.seed(42)


best_solution, best_score = simulated_annealing(
    objective=rastrigin,
    bounds=bounds,
    step_size=0.5,
    initial_temp=100.0,
    cooling_rate=0.95,
    min_temp=1e-3,
    iterations_per_temp=100
)


print("Best solution:", best_solution)
print("Minimum value:", best_score)
```

```
Best solution: [-0.0017892009890418947, 0.03207996747483055]
Minimum value: 0.20411837507225172
```